

Canceling async Redis command leaves connection open, in unsafe state for future commands #2665

Closed

#2674

drago-balto opened on Mar 26, 2023

Version: 4.5.3**Platform:** Python 3.8 on Ubuntu / Generic**Description:** Canceling async Redis command leaves connection open, in unsafe state for future commands

This is a reincarnation of [#2624](#), which was closed with an incomplete fix and a possibly unreliable test case. This is the same issue that recently got a lot of attention due to ChatGPT outage, and that remains only partially fixed. The cancellation shielding introduced in [#2641](#) addressed only the cancellation of Redis pipeline operation, but non-pipelined ops are still vulnerable.

This time I am attaching a script that reproduces the issue reliably without relying on an external, slow Redis server. This is achieved by inserting a small TCP socket proxy between the Redis client and local Redis server, with the proxy introducing a 0.1 second delay when sending data in either direction.

Running this script with a Redis server running locally on port 6379 produces the following output:

```
$ python redis_cancel.py
managed to cancel the task, connection is left open with unread response
bar: b'foo'
ping: False
foo: b'PONG'
```

```
import asyncio

from redis.asyncio import Redis

async def pipe(reader: asyncio.StreamReader, writer: asyncio.StreamWriter, delay: float, name: str):
    while data := await reader.read(1000):
        # print(name, 'received:', data)
        await asyncio.sleep(delay)
        writer.write(data)
        await writer.drain()

class DelayProxy:
```

```

def __init__(self, addr, redis_addr, delay: float):
    self.addr = addr
    self.redis_addr = redis_addr
    self.delay = delay

async def start(self):
    server = await asyncio.start_server(self.handle, *self.addr)
    asyncio.create_task(server.serve_forever())

async def handle(self, reader, writer):
    # establish connection to redis
    redis_reader, redis_writer = await asyncio.open_connection(*self.redis_addr)
    pipe1 = asyncio.create_task(pipe(reader, redis_writer, self.delay, 'to redis:'))
    pipe2 = asyncio.create_task(pipe(redis_reader, writer, self.delay, 'from redis:'))
    await asyncio.gather(pipe1, pipe2)

async def main():

    # create a tcp socket proxy that relays data to Redis and back, inserting 0.1 seconds of
    dp = DelayProxy(addr=('localhost', 6380), redis_addr=('localhost', 6379), delay=0.1)
    await dp.start()

    # note that we connect to proxy, rather than to Redis directly
    async with Redis(host='localhost', port=6380) as r:

        await r.set('foo', 'foo')
        await r.set('bar', 'bar')

        t = asyncio.create_task(r.get('foo'))
        await asyncio.sleep(0.050)
        t.cancel()
        try:
            await t
            print('try again, we did not cancel the task in time')
        except asyncio.CancelledError:
            print('managed to cancel the task, connection is left open with unread response')

        print('bar:', await r.get('bar'))
        print('ping:', await r.ping())
        print('foo:', await r.get('foo'))

if __name__ == '__main__':
    asyncio.run(main())

```

8

4

drago-balto on Mar 26, 2023

Author ...

@chayim Please take a look. Let me know if there is anything I can do to help.



chayim mentioned this on Mar 26, 2023

✓ Off by 1 - Canceling async Redis command leaves connection open, in unsafe state for future commands #2624

[@drago-balto](#) thank you for the targetted report, we really appreciate this! My concern is this both being reproducible, and fixed across all async connections, hence loving this report ♥

Consolidating here.



[chayim](#) mentioned this [on Mar 26, 2023](#)

✓ [CVE Request for Async Connection Bugs #2664](#)

🔗 [Fixing cancelled async futures #2666](#)

chayim on Mar 26, 2023

Contributor ...

For anyone tracking this - please feel free to weigh in on the pull request [@dvora-h](#) and I have started on. Note the beginning of test coverage. I'd welcome any help that's available - as we have an *amazing* community.

Thank you [@drago-balto](#) specifically. Your code sample above is/was a great trigger.

adamserafini on Mar 27, 2023

...

Will `aioredis 2.x.x` get updated as well? I presume the issue will affect that library as well (as the asyncio code was ported into this library from there).

chayim on Mar 28, 2023

Contributor ...

[@adamserafini](#) I don't think so... this has been the primary library for a little over a year....

auvipy on Mar 28, 2023

...

so which versions will get the proper fix? will 4.3.x version get the backport fix?



[auvipy](#) mentioned this in 2 pull requests [on Mar 28, 2023](#)

🔗 [Update redis.txt 4.3.6 celery/celery#8161](#)

🔗 [Update redis.txt to 4.3.6 to fix severe issue celery/kombu#1676](#)



[bmerry](#) added 2 commits that reference this issue [on Mar 29, 2023](#)

Bump redis to latest version ...

0037c55

Bump redis to latest version ...

ce4d4f2



[bmerry](#) mentioned this [on Mar 29, 2023](#)

🔗 [Bump redis to latest version ska-sa/katgpucbf#539](#)

It'll be backport to 4.3 and 4.4.

[@dvora-h](#) can you include this issues # in each of the branch tickets that are about to be released.

♥ 2

18 remaining items

Load more

drago-balto on Apr 3, 2023

Author ...

[@yangbodong22011](#) we can't yet assume that. From my standpoint 4.4 and 4.5 contain the fix. I'd like to *also* get this into 3.6.

The plan is to use [@drago-balto](#) amazing async skills and the tests, to craft a 4.3.x valid set of tests that let everyone sleep at night.

I agree with the above. Just because this *one* test did not trigger an issue with Python 3.6 + Redis 4.3.x does not automatically mean all is well. I don't know enough about the library and the differences between 4.3 and 4.4/4.5 lines to offer more specific advice, but what [@chayim](#) says resonates with me.

auvipy on Apr 4, 2023

...

a new release of 4.3.x with the fix is needed for celery project as it has many python versions



[yangbodong22011](#) added a commit that references this issue [on Apr 10, 2023](#)

Upgrade redis-py to 4.4.4 and above ...

9640ee7



[jguillon](#) mentioned this [on Apr 19, 2023](#)

[chore\(deps\): update redis Flared/dramatiq-abort#20](#)

agronholm on Apr 19, 2023

Contributor ...

Is this the same issue I'm encountering here, or is mine different?

```
import asyncio
from asyncio import CancelledError, create_task, sleep

from redis.asyncio.client import Redis

async def push_events() -> None:
```



```

async with Redis() as redis:
    task = create_task(redis.blpop(["dummy_topic"]))
    await sleep(0.5)
    task.cancel()
    try:
        await task
    except CanceledError:
        pass

asyncio.run(push_events())

```

Result:

```

Task exception was never retrieved
future: <Task finished name='Task-3' coro=<Redis._try_send_command_parse_response() do...
defined at /usr/local/lib/python3.11/site-packages/redis/asyncio/client.py:503>
exception=ConnectionError('Connection closed by server.')>
Traceback (most recent call last):
  File "/usr/local/lib/python3.11/site-packages/redis/asyncio/client.py", line 505, in
_try_send_command_parse_response
    return await conn.retry.call_with_retry(
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.11/site-packages/redis/asyncio/retry.py", line 62, in
call_with_retry
    await fail(error)
  File "/usr/local/lib/python3.11/site-packages/redis/asyncio/client.py", line 501, in
_disconnect_raise
    raise error
  File "/usr/local/lib/python3.11/site-packages/redis/asyncio/retry.py", line 59, in
call_with_retry
    return await do()
        ^^^^^^^^^^^^^
  File "/usr/local/lib/python3.11/site-packages/redis/asyncio/client.py", line 488, in
_send_command_parse_response
    return await self.parse_response(conn, command_name, **options)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.11/site-packages/redis/asyncio/client.py", line 544, in
parse_response
    response = await connection.read_response()
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.11/site-packages/redis/asyncio/connection.py", line 840,
in read_response
    response = await self._parser.read_response(
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.11/site-packages/redis/asyncio/connection.py", line 264,
in read_response
    response = await self._read_response(disable_decoding=disable_decoding)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.11/site-packages/redis/asyncio/connection.py", line 272,
in _read_response
    raw = await self._readline()
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.11/site-packages/redis/asyncio/connection.py", line 348,
in _readline
    raise ConnectionError(SERVER_CLOSED_CONNECTION_ERROR)
redis.exceptions.ConnectionError: Connection closed by server.

```

That a task's exception was never retrieved speaks of a broader problem.



catalinm-am mentioned this [on Apr 21, 2023](#)



[redis-py Race Condition vulnerability feast-dev/feast#3603](#)

Chronial on May 8, 2023

Contributor ...

Duplicate of [#2499](#)?



Chronial mentioned this [on May 8, 2023](#)



[BaseException at I/O corrupts Connection #2499](#)

kristjanvalur on May 8, 2023

Contributor ...

This is hopefully fixed with [#2695](#)

agronholm on May 8, 2023

Contributor ...

This is hopefully fixed with [#2695](#)

It looks like it would be.



darakian mentioned this [on May 31, 2023](#)



[\[GHSA-8fww-64cx-x8p5\] redis-py Race Condition due to incomplete fix github/advisory-database#2335](#)

chayim on Jun 27, 2023

Contributor ...

Should have been autoclosed. Closing.



chayim closed this as [completed](#) [on Jun 27, 2023](#)



github-actions mentioned this in 2 issues [on May 2, 2024](#)



[MEDIUM - redis: Async command information disclosure FabioBartoli/Linuxtips-telegram-communication#27](#)



[MEDIUM: redis - redis: Async command information disclosure FabioBartoli/Linuxtips-telegram-communication#41](#)

Sign up for free

to join this conversation on GitHub. Already have an account? [Sign in to comment](#)

Assignees

No one assigned

Labels

No labels

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

 Code with agent mode

▼

 **Version 4.5.4**
redis/redis-py

Participants



+4